Exploring DeepFakes: Face Reenactment Report

Yusuf Arslan, Erdem Çelik, Mehmet Yavuz Katrancı, Nahit Eren Bulut

12.10.2026

1 Introduction

This project studies face reenactment, one way to make a deepfake. The aim is to take the face from one picture and let it follow the movement from another source, then write the result as a video. The assignment asked for a working deepfake model, or any model with a comparable face reenactment or identity-mapping effect. It also asked us to document every choice and turning point, to include the failed experiments with the reasons for the failure, and to reflect on what each experiment taught us. This report gives that record.

Every method in this report works with two inputs. The identity image supplies the face. The driving or motion source supplies the movement. The two roles never change, but each method uses the pair its own way, and the chapters give the inputs their own names. The autoencoder chapter calls them the source face image and the target video. The FOMM chapter calls them the source image and the driving video. The rectified flow chapter pairs a portrait with a body frame from the clip.

We explored three methods, one from each family. The first is an autoencoder with one shared encoder and one decoder per person. We built and trained it ourselves to learn the basics, and we also tested three pretrained face-swap models from FaceFusion. The second is the keypoint-based First Order Motion Model (FOMM). It needs no landmark detector and no 3D face model, and it animates a source photo with the motion of a driving video. The third is a head swap with a rectified flow model and a LoRA adapter. A released checkpoint that needs only four sampling steps fits the one laptop with the 4 GB GPU that we had.

The three methods differ inside, but they share one picture-level pipeline. They read an identity image and a motion source, run a model that writes one output frame per frame of the motion source, and save the frames as a clip. The autoencoder had to be trained first. FOMM and the rectified flow pipeline load released checkpoints, and the rectified flow pipeline adds restoration, smoothing and reassembly after generation. We judge the frames by eye and with the measures defined in each chapter.

Section 2 covers the autoencoder experiments and the pretrained FaceFusion models, Section 3 covers the GAN background and FOMM, and Section 4 covers the rectified flow pipeline. Section 5 compares the three methods side by side, and the conclusion chapter reports what worked, what failed, and what we learned.

2 Autoencoders

2.1 Introduction & Problem Definition

The goal of our project is swapping faces between two members of the group while keeping the source identity, regenarate the target video with the target's mimic and movements meanwhile trying to keep the video's quality. This document will explain one of there alternative architectures: autoencoders.

Dataset	Prepared face images	Resolution
Person A	100	128×128
Person B	138	128×128
Total	238	128×128

Table: Dataset

We first decided to build a small auto encoder first by ourselves before trying pretrained models to experience and learn basics. We then tried pretrained face-swapping models and compared their outputs with our own approach. We covered full process, from preparing the images and training a model to generating videos and looking closely at what worked and what did not.

2.2 Dataset Preparation & Preprocessing

2.2.1 Data collection and frame extraction

We recorded short videos of Person A and Person We saved every fifth frame instead of every frame, in order to avoid collecting too many nearly identical images.

2.2.2 Face detection and image preparation

Next, we detected faces with OpenCV's Haar Cascade classifier. We converted frames to grayscale for detection. We also added a 25% margin around each face before cropping, so the images included some surrounding facial features.

Images were resized to 128×128 pixels. During training, PyTorch's ToTensor transformation scaled the pixel values to the 0–1

2.3 Autoencoder Architecture

We built the model using one Encoder and two separate Decoders. Decoder A was trained on Person A's images and Decoder B on Person B's. The Decoders had the same design, but they learned their own parameters.

The Encoder had four convolutional layers. The Decoder reversed this process with four transposed-convolution layers. We used Sigmoid activation kept the output pixel values between 0 and 1.

First, we sent encoders's representation to the Decoder for the same person to reconstruct the input face. To try face swapping, secondly, we used the other person's Decoder: A through Decoder B, and B through Decoder A.

2.4 Model Training

In each epoch, we first trained on Person A's images with Decoder A and then on Person B's images with Decoder B. For each run, the model calculated the reconstruction error, updated the shared Encoder and the relevant Decoder.

We trained for 50 epochs and saved the model weights. We then loaded those weights and continued for another 100 epochs.

At epoch 150, Person A's average training MSE was 0.002935 and Person B's was 0.001866.

Epoch	Person A MSE	Person B MSE
146	0.003046	0.001939
147	0.003013	0.001929
148	0.003014	0.001905
149	0.002996	0.001882
150	0.002935	0.001866

Table 1: Recorded training results

Experiment	A → B output	B → A output
50 epochs	deepfake_A_to_B.mp4	deepfake_B_to_A.mp4
150 epochs	deepfake_150_A_to_B.mp4	deepfake_150_B_to_A.mp4

Table: Experiments

Model	Output size	Frames	Duration
HyperSwap 1A 256	576 × 1024	576	~19.25 s
InSwapper 128	576 × 1024	576	~19.25 s
SimSwap 256	576 × 1024	576	~19.25 s

Table: Pretrained models

2.5 Experiments & Results

2.5.1 Video generation and comparison

We generated videos after both 50 and 150 epochs. We tested A → B and B → A using generate_video.py and generate_video_150.py.

2.5.2 Observations

When we compare the videos we could not see a clear improvement between 50 and 150 epochs. In addition to that, the generated A → B face did not look like Person B, and the B → A result did not look like Person A. In other words, our model was learning reconstruction but not performing the identity transfer we wanted.

We think the main issue was the training objective. The model learned to recreate the original face instead of swapping it with another face.Our small dataset and may also have played a part, although we did not test those factors separately. This unsuccessful experiment helped us decide what to try next.

2.6 Experiments with Pretrained Face-Swapping Models

After the autoencoder experiments, we tested three pretrained models in FaceFusion: HyperSwap 1A 256, InSwapper 128 and SimSwap 256. We used a source face image and a target video, with the source providing the identity and the target providing the expressions and movement. We ran the models for inference and did not retrain them

HyperSwap and InSwapper looked fairly natural and gave similar results. SimSwap followed the facial movements too, but we noticed more visible differences around the eyes, the middle of the face and the skin texture.

This was a visual comparison, but it was clear that the pretrained models gave more promising results than our own autoencoder.

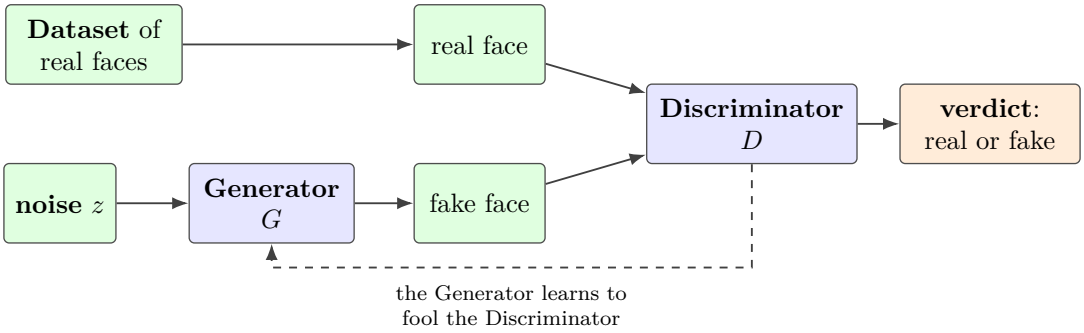


Figure 1: The training loop of a standard GAN. The Discriminator compares each fake face with real faces and gives a verdict; the dashed feedback teaches the Generator to fool it.

2.7 Evaluation & Lessons Learned

2.7.1 Overall evaluation

Training and producing videos was not successfull, however. FaceFusion gave better results with HyperSwap and InSwapper. For us, the difference showed how important it is to use a pretrained model for a spesific task.

We had only 100 images of Person A and 138 of Person B, it was a limitation.

2.7.2 Lessons learned

The biggest lesson was that a low loss does not mean a usable result. Our model was getting better in reconstructuring faces but not good at the same as swapping the faces. Training for another 100 epochs did not fix this.

As a sumary, we learned much more from the project than how to run a face-swapping tool. Building our own model helped us understand preprocessing, training and evaluation. Using pretrained models afterwards, gave us a chance to compare how it would improve the result

3 GAN Background and Keypoint-based FOMM

A generator is like a painter who draws faces. A discriminator is like an expert who judges each face as real or fake and teaches the painter. This game cannot follow orders, however. For reenactment we must keep one identity and copy the motion from a video. We used the First Order Motion Model (FOMM). FOMM is a keypoint-based method, and the released configuration trains no adversarial loss.

3.1 Standard GANs

A standard GAN holds two networks. The generator G reads a noise vector z and draws a face; the discriminator D reads an image and gives a verdict: real or fake. The two train against each other, so the generator learns to fool the discriminator (Goodfellow et al., 2014), as Figure 1 shows.

We cannot control the output. Identity, pose and mood are all mixed into the noise input. We cannot say “keep this person, copy that motion”. A reenactment model needs two inputs instead: a source image (who) and a driving video (how it moves).

3.2 First Order Motion Model

FOMM animates one source photo with the head motion and the facial expressions from a driving video. The method comes from Siarohin et al. (“First Order Motion Model for Image

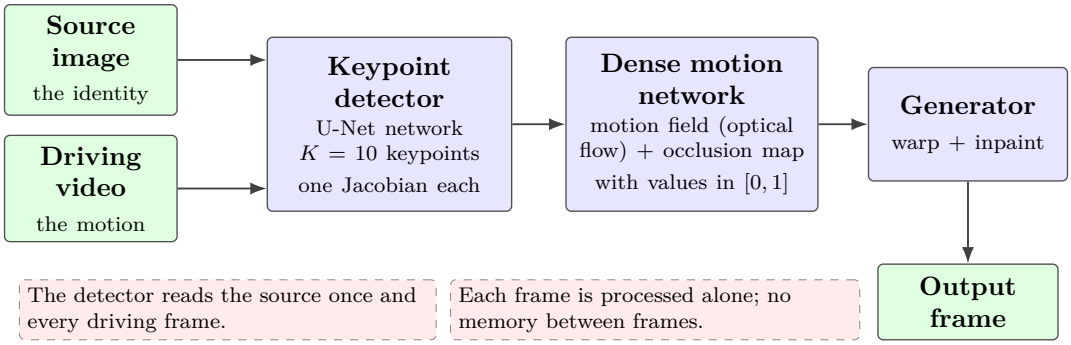


Figure 2: The First Order Motion Model pipeline: keypoints with Jacobians, a motion field and an occlusion map, then warp and inpaint. Each frame is processed alone.

Animation", NeurIPS 2019). It needs no landmark detector and no 3D face model. It learns its own keypoints per object category; our category is faces. It follows Monkey-Net, which the paper calls the first object-agnostic deep model, and FOMM improves on it.

3.2.1 First-order motion

0th order motion moves a point only in x and y, so the skin keeps its shape and the result looks stiff and robotic. 1st order motion also measures how the skin around the point turns, grows and stretches, through a small 2×2 Jacobian matrix per keypoint. The paper calls it a first order Taylor expansion.

3.2.2 The three parts

The pipeline has three parts (Figure 2).

The **keypoint detector** is a U-Net network (the paper's word; the code names the class Hourglass). It finds $K = 10$ keypoints for the faces model and one Jacobian per keypoint. The expected point position is a weighted average over the output heatmaps (soft-argmax). An equivariance loss teaches the detector during training, so it needs no keypoint labels.

The **dense motion network** turns the sparse keypoints into a dense motion field (optical flow) and an occlusion map with values between 0 and 1.

The **generator** encodes the source image into features and warps these features with the flow. The code multiplies the warped features by the occlusion map (`out = out * occlusion_map`). Where the map is near 0, the warped features lose their weight, and the decoder reconstructs that area from learned appearance priors. The decoder produces the whole output frame from the weighted features.

Each frame is processed on its own. No state passes from frame to frame (no recurrence).

This configuration trains no adversarial loss. `vox-256.yaml` sets `generator_gan: 0`. The training loop then skips the discriminator update completely, and the feature-matching loss never runs, although its weights are not zero. The active generator losses are perceptual reconstruction, equivariance value, and equivariance Jacobian.

3.2.3 Occlusion and inpainting

A closed mouth or a turned head reveals pixels that do not exist in the source photo (self-occlusion). The occlusion map O is a learned per-pixel weight on the warped features. Where O is near 0, the model trusts the warped features less, and the decoder fills more of the area from learned appearance priors (Figure 3). The decoder does not copy a literal average tooth image.

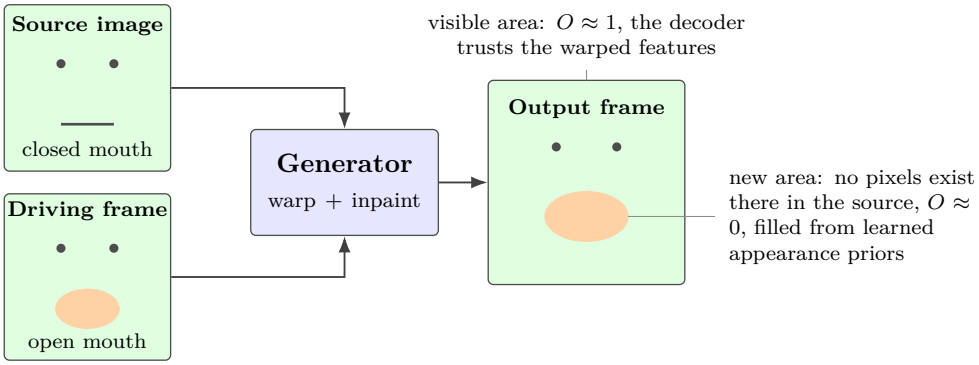


Figure 3: Occlusion and inpainting. O is a learned per-pixel weight on the warped features. Where O is near 0, the model trusts the warped features less, and the decoder fills the area from learned appearance priors.

Item	Value
Checkpoint	`vox-cpk.pth.tar` (VoxCeleb)
Source portrait	512 × 512
Driving clip	256 × 256, 320 frames, 29.97 FPS
Mode and flags	CPU, `--relative`, `--adapt_scale`
Output	256 × 256, 320 frames, 29.97 FPS, 10.7 s
Speed	about 2.05 it/s, about 2.5 minutes

Table 2: The local FOMM run: setup, flags and output.

Frame	Time	Driving mouth	Result (visual reading)
10	0.3 s	small shift	braces kept
195	6.5 s	lips close	mouth hidden
288	9.6 s	wide smile	braces lost, teeth plain

Table 3: The braces test: three frames and our visual reading of the output. We stored no occlusion measurements for these frames.

These are the failure modes we saw. Rare personal details are lost in inpainted areas. Very large head turns, about 45 degrees or more, leave blurry patches. Both limits come from our own observation, not from the paper.

3.3 Local experiment

We ran the released VoxCeleb checkpoint `vox-cpk.pth.tar` in CPU mode on an Apple Silicon machine. The source portrait is 512x512; the driving clip is 256x256, 320 frames, 29.97 FPS. With `--relative` the keypoints move relative to the first frame; `--adapt_scale` adjusts the motion size with the convex hull of the keypoints.

The output is 256x256, 320 frames, 29.97 FPS and 10.7 seconds long. The recorded speed is about 2.05 iterations per second, about 2.5 minutes in total. Head turns and blinks transferred smoothly; mouth motion needed inpainting. Table 2 lists the setup and the output.

3.4 Failure mode: braces

The braces disappear in some output frames (Table 3). We treat the cause as a hypothesis: the training videos may show no braces, so the learned appearance priors cannot reconstruct them. We have no direct evidence for this hypothesis. The repository holds no analysis of the dataset content, and the official VoxCeleb description states no such exclusion.

FOMM gave the project a fast per-frame method. Every output frame comes from the decoder, which reads the occlusion-weighted warped features. Rare details such as braces disappear in some outputs, and we treat the cause as a hypothesis. There is no memory across frames, so the method cannot repair a frame from its neighbours.

4 Rectified Flow Head Swap

4.1 Scope and hardware

We wanted to swap a video face with one from a selfie. The run had to stay on one laptop with a 4 GB NVIDIA GPU and 8 GB of system RAM. We sent no data to a cloud service.

The output had to keep the source motion, expression, lighting and background. Only the head should change. The clip is 96 frames at 512x512 and lasts 3.2 seconds.

4.2 How we measured

Every number comes from one script, run once per variant, each on the same 96 frames (175 to 270). The results table reports it. The raw run records, the metric CSV and the 96-frame sets are not in the repository, so the reported numbers cannot be re-derived from this checkout.

Face change is a temporal change indicator: the mean absolute grey difference between two frames inside one face box. The code takes the box from the first frame of the pair and reuses it for the second frame, so the box is not aligned to the second frame. Motion, lighting, texture change and flicker all enter the value, and the value has no calibrated threshold. The unswapped footage sets the control level.

Consecutive identity is how similar a face is to the one before it, measured in a face recognition embedding, so a high value means temporal similarity; a wrong identity that stays consistent also scores high. Identity to the portrait measures similarity to the reference portrait; the unswapped footage scores near zero because it is a different person. Hair noise is how much the hair region flickers, and lower is better; p95 marks the worst frames, the value 95 percent stay under.

4.3 Pipeline overview

The flow has four stages. Generation transfers the identity, restoration cleans each face with GFPGAN, smoothing removes the boil, and reassembly attaches the audio. Only the last stage writes a video file.

The boil is the flicker that comes from generating each frame on its own. Figure 4 shows the chain in order, with the cost of each stage.

4.4 How a rectified flow model works

A rectified flow model learns one job: connect a clean image and noise with a straight line. Training picks one random time per example and mixes the two ends at that time. The model predicts the direction that points from the noise end to the image end. The guess is compared with the true direction along the line, and the weights are then nudged.

At sampling time the same model starts from pure noise. Each Euler step moves the sample a small way along the predicted direction, and an image forms. The sampling code calls these steps denoising steps, because the sample becomes less noisy at each step. Figure 5 draws training and sampling side by side.

We did not train the base model; we used a released checkpoint. The released model is step distilled, so four steps are enough for it. We could not tell the difference between four steps and eight.

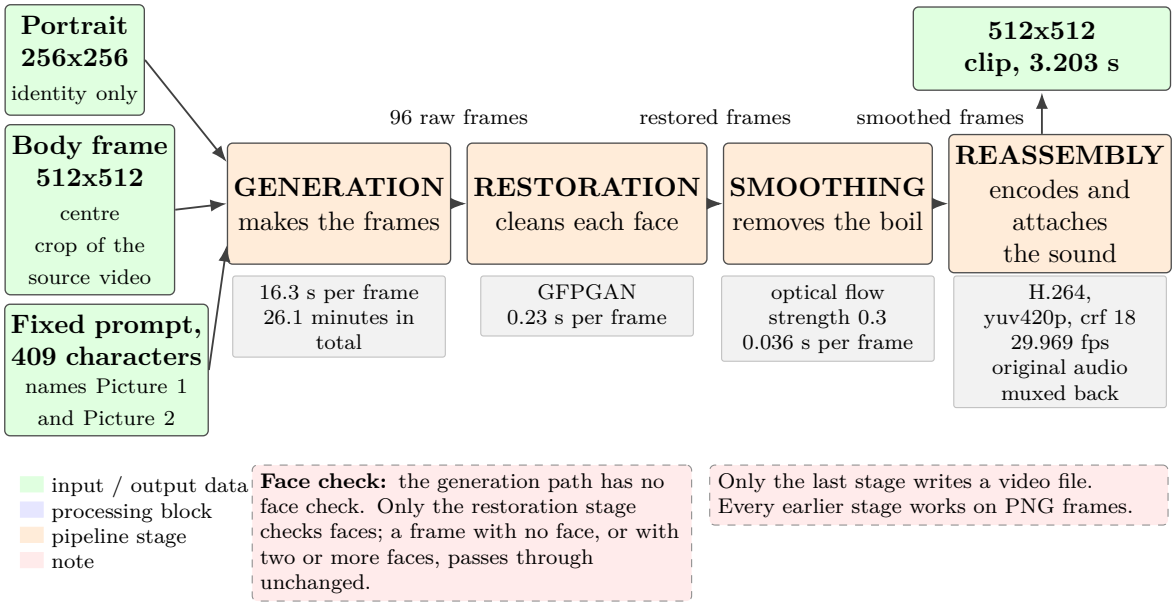


Figure 4: The four-stage pipeline: generation, restoration, smoothing and reassembly, with the cost measured for each stage.

4.5 The model and the LoRA adapter

The model is a stack of transformer blocks (Figure 6). A latent token of 128 channels is projected to width 3072. It passes through five double-stream blocks, then 20 single-stream blocks and a final layer. In the first part the text and image tokens stay apart, and they meet through joint attention.

Later the two streams are joined into one. The final layer projects back to 128 channels for the decoder.

The adapter is a small pair of matrices, A and B, trained beside each frozen weight. The adapter attaches inside the transformer blocks. It takes the attention projections and the feed-forward linears, and in the single-stream blocks the two fused linears. The block output adds a scaled product of A and B to the frozen term. Only A and B are trained, so the base weights never change. The adapter carries the head swap behaviour.

4.6 Generation

Generation (Figure 7) covers conditioning, sampling and decoding. A fixed prompt of 409 characters names Picture 1 as the body and Picture 2 as the head.

Qwen3-4B encodes the prompt once per session on the CPU, about 80 seconds, and its memory is then freed. Both pictures become VAE latents: the 512x512 body crop a 64x64x32 grid of about 1024 tokens, the 256x256 portrait about 256 tokens.

FLUX.2-klein-4B renders each frame in four sampling steps. Each step moves the latent along the direction the model predicts. The sampler calls the model once per step and feeds the result back. The text embeddings and the two image latents guide every step. A community-trained LoRA adapter of 80 adapted weights carries the head swap.

The same VAE decodes the final latent into pixels. Picture order is part of the adapter contract: the adapter's own documentation names Picture 1 the body and Picture 2 the head, and says not to reverse the images. A reversed order gives the wrong head.

The generator runs as a 4-bit GGUF file. The production loop calls the generator once per frame and holds no face gate, and no multi-face check exists on the generation path. The only face check sits in the restoration stage: the gate passes a frame only when it detects exactly

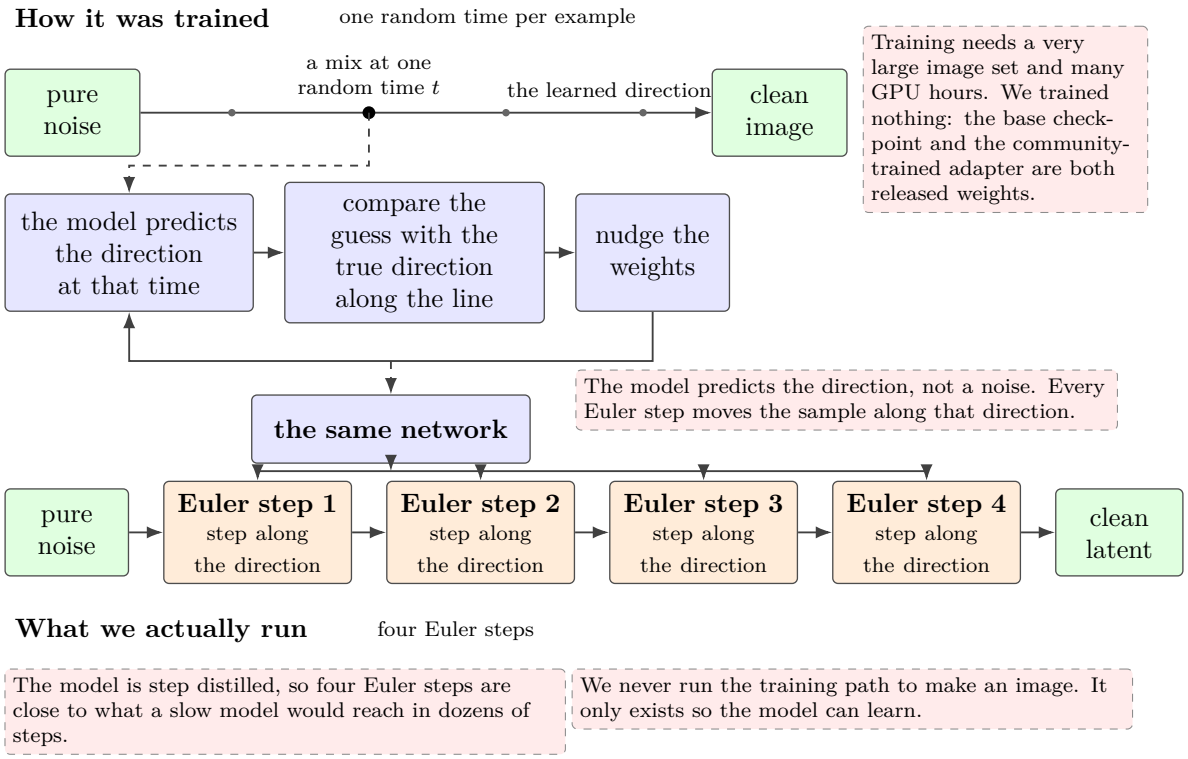


Figure 5: The rectified flow model in training and at sampling. Training draws one random time per example, mixes the image and noise at that time, and teaches the model the direction along the straight line between the two ends. Sampling starts from noise and takes four Euler steps along the predicted direction. Both jobs use one and the same network.

one face. Frames with no face or with two or more faces are copied through unchanged.

The output is 96 frames from 5.8 to 9.0 seconds of the source, all with one fixed seed. Each frame took 16.3 seconds, so the stage used 26.1 minutes.

4.6.1 Adapter version comparison

The recorded clip uses adapter v1 throughout. A newer adapter, v1.1, was released during the work, and we tested two prompt variants, A and B. Table 4 compares the two versions.

v1.1 ran, but it took about 199 seconds per frame against 16.34 for v1. Two measured values rose with it: the adapter rank, from 128 to 512 (four times larger), and the peak allocated memory, from 3.60 GiB to 4.68 GiB, past the 4.0 GiB of the card. That peak value is a PyTorch allocation counter (`torch.cuda.max_memory_allocated`), not driver-level or hardware evidence. Memory pressure is a plausible cause of the slowdown, but the measurements do not settle it. The extra time was not worth it for us.

We judged the look by eye and preferred v1, so the clip kept the v1 output. No metric script was run on v1.1, so every other number here comes from v1. The grids in Figure 8 are the only evidence.

4.7 Face restoration

GFPGAN v1.4 cleans every frame (Figure 9). It detects and aligns the face, runs a restoration network, and composites the crop back. The weights are official, but we built the network in project code because the published wrappers do not import.

The stage costs 0.23 seconds per frame. The frames looked cleaner to us, and the mean face change came down, though the tail improved less.

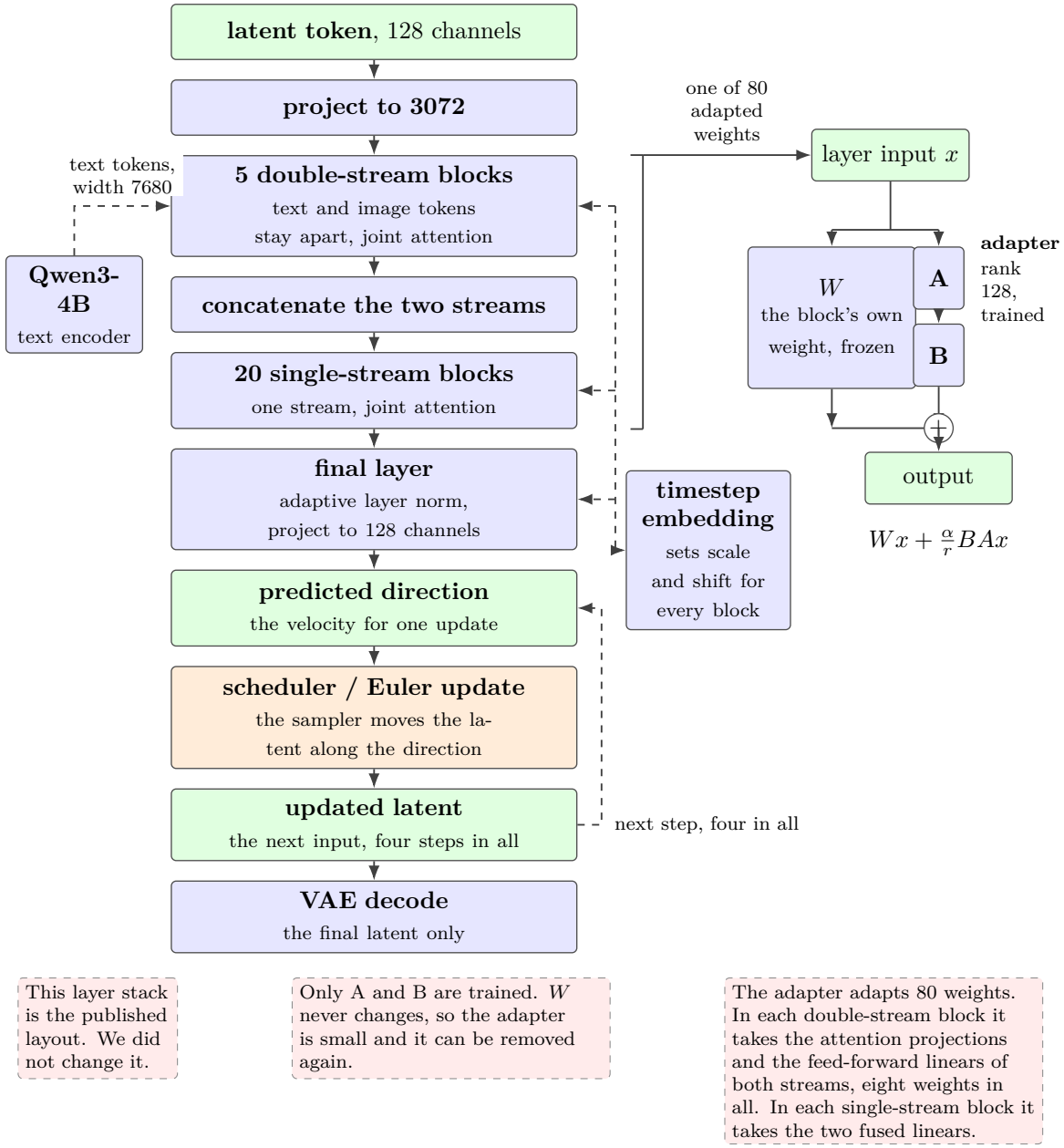


Figure 6: The model path: the latent token is projected to width 3072, passes the double-stream and single-stream blocks, and the final layer predicts a direction, the velocity for one update. The sampler applies the update, and only the final latent reaches the VAE decode. The text encoder and the timestep embedding enter as conditioning. The bracket on the right marks one weight inside a block, where a small trained adapter sits beside the block's own frozen weight.

Hair noise fell slightly, a small gain on its own. The fidelity weight does nothing: the v1.3.4 code passes no weight argument, and newer versions accept it without reading it. Restoration sees one frame at a time, so it does not guarantee temporal consistency.

4.8 Temporal smoothing

The smoothing stage compares frames with each other and removes the boil (Figure 10). The per-pixel deflicker averages each pixel with its temporal neighbours, and it is too strong. It costs 0.007 seconds per frame.

The deflicker pushes the face change below the unswapped level. The value has no calibrated

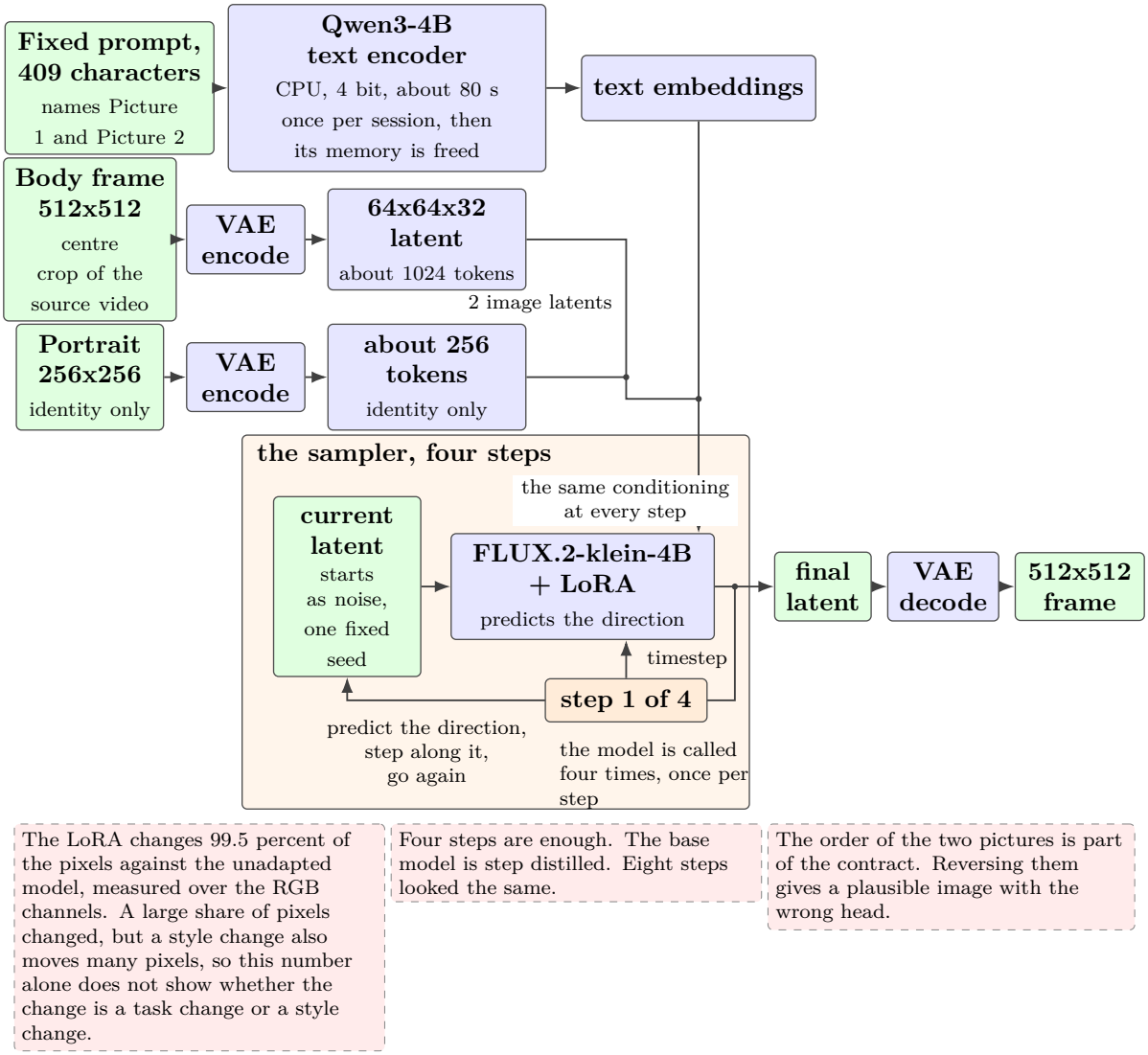


Figure 7: Inside the generation stage: Qwen3-4B encodes the prompt once, and the sampler then calls the model four times, once per step. Each call receives the current step number and the same conditioning, and each predicted direction drives one Euler update of the latent. The final updated latent is decoded into a 512x512 frame.

	v1 (recorded)	v1.1 (new)
Adapter rank	128	512
Seconds per frame	16.34	about 199, twelve times slower
Peak VRAM allocated	3.60 GiB	4.68 GiB, past the 4.0 GiB of the card
Peak system RAM	5.48 GiB	5.3 to 5.5 GiB
Cost for 96 frames	26 minutes	about 5.3 hours

Table 4: Adapter v1 against v1.1 on the same machine; only v1 was measured with the metric script.

threshold, and motion, lighting, texture change and flicker all enter it, so this drop alone does not show that real motion went away. Optical flow estimates the dense motion between neighbouring frames and warps them onto the current one. The warp follows surface points across time, but optical flow does not guarantee temporal consistency. It costs 0.036 seconds per frame.

The control sets the reference change level at 6.66, and the distance column is the face

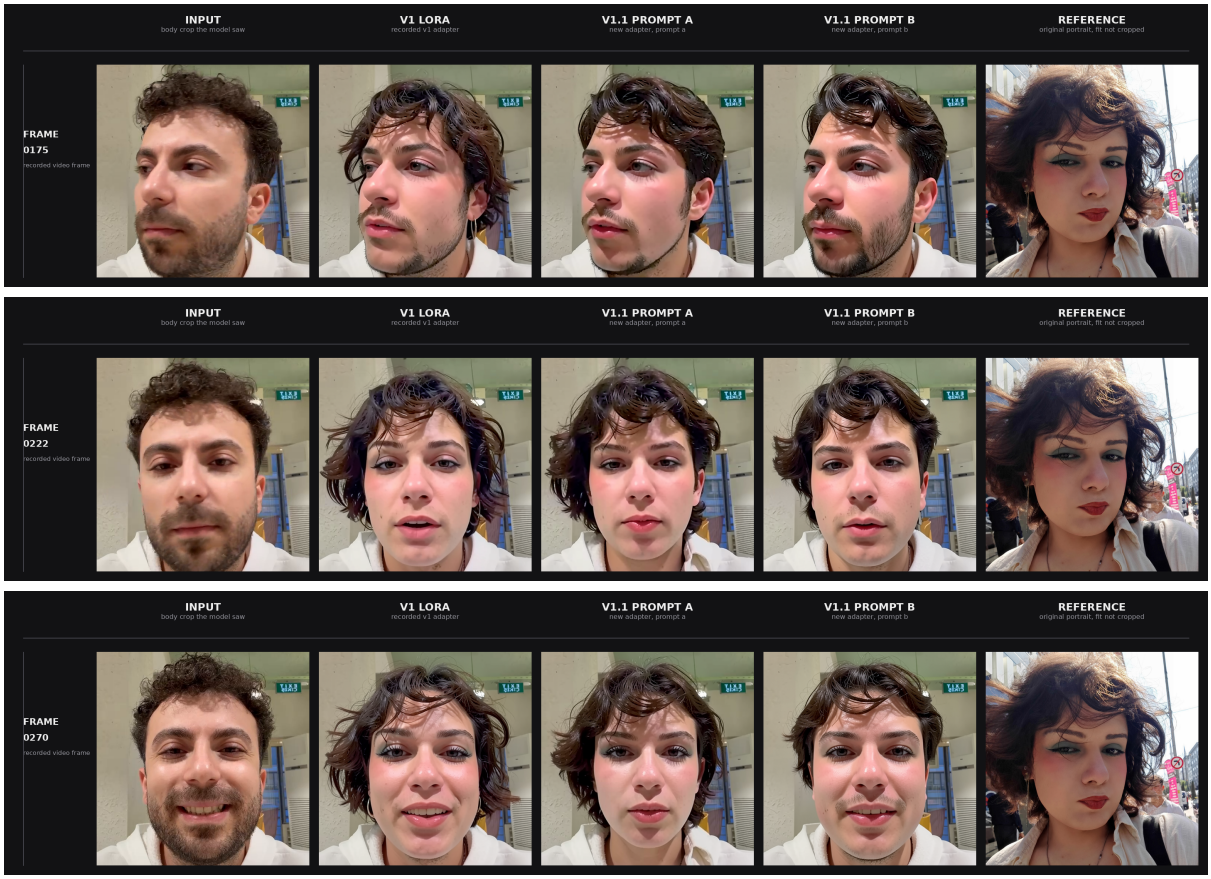


Figure 8: Each row is one source frame, the frame number printed inside. The five columns run from the input body crop through the v1 adapter, v1.1 with prompt A, v1.1 with prompt B, to the reference portrait; only the adapter version and the prompt variant change.

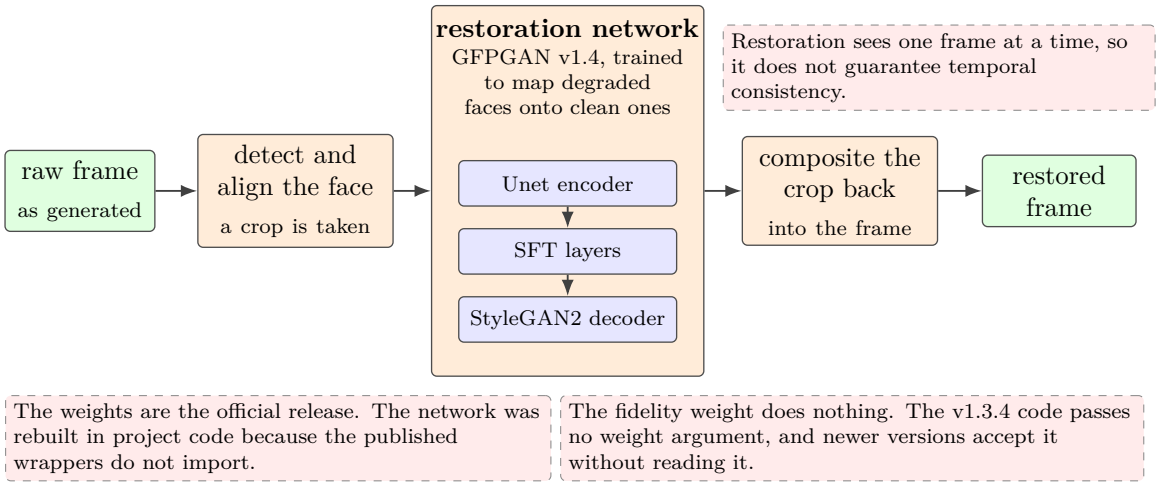


Figure 9: The face restoration stage. Each raw frame is taken on its own: the face is detected and aligned, a crop runs through the restoration network, and the restored crop is composited back into the frame.

change minus that level. At strength 0.3 the chained pass comes close to the control level, and the raw pass stays above it. A stronger setting sits below the control; the drop cannot separate removed motion from removed flicker.

The recommended output is the chained set at strength 0.3. It beats the flow-only variant

Frames smoothed	Strength	Face change	Distance from the control	p95	Consecutive identity	Hair noise
Raw	0.3	6.9599	+0.3034	16.2944	0.9330	32.2358
Raw	0.6	6.6663	+0.0098	15.6581	0.9391	32.0603
Restored, then flow	0.3	6.6303	-0.0262	15.2802	0.9377	31.7306
Restored, then flow	0.6	6.3071	-0.3494	14.7575	0.9431	31.5448

Table 5: The smoothing sweep: two frame sets at two strengths, all measured on the same 96 frames.

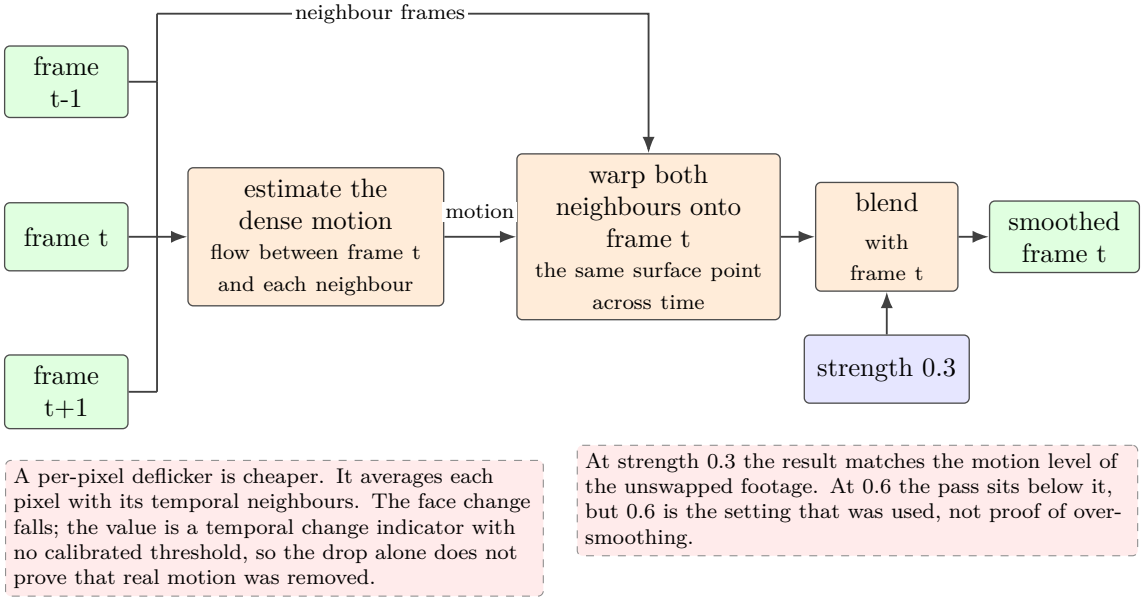


Figure 10: The temporal smoothing stage. Each frame is compared with its two neighbours: the dense motion is estimated, the neighbours are warped onto frame t and the three are blended at strength 0.3.

on face change, on the tail and on hair noise. Only consecutive identity is slightly lower. The deflicker reaches its low face change at a value below the control; that value cannot separate removed motion from removed flicker, so we kept flow.

4.9 Reassembly

The encoder turns a frame set into a playable clip (Figure 11). It writes H.264 at 29.969 fps and trims both streams. The audio must come from the same interval as the frames, 5.8 to 9.0 seconds of the source. The reassembly tool derives that start from the frame constants, so a normal run takes the audio from 5.839 seconds, and `--audio-start` overrides it. An earlier version took the full-video audio from t=0, which mixed the wrong interval into the picture. The `-shortest` flag only cuts the audio to the video length; it does not shift the start. Its output is a 512x512 clip of 3.203 seconds.

The encode is lossy: it changes pixels slightly, but it cannot alter the face motion. In the raw swapped set the face boils, and the results table holds the values.

4.10 Results

All numbers come from one script, run once per variant, each on the same 96 frames (175 to 270). Lower hair noise is better, and higher identity values are better. Face change is a temporal change indicator with no calibrated threshold. A value below the control level shows

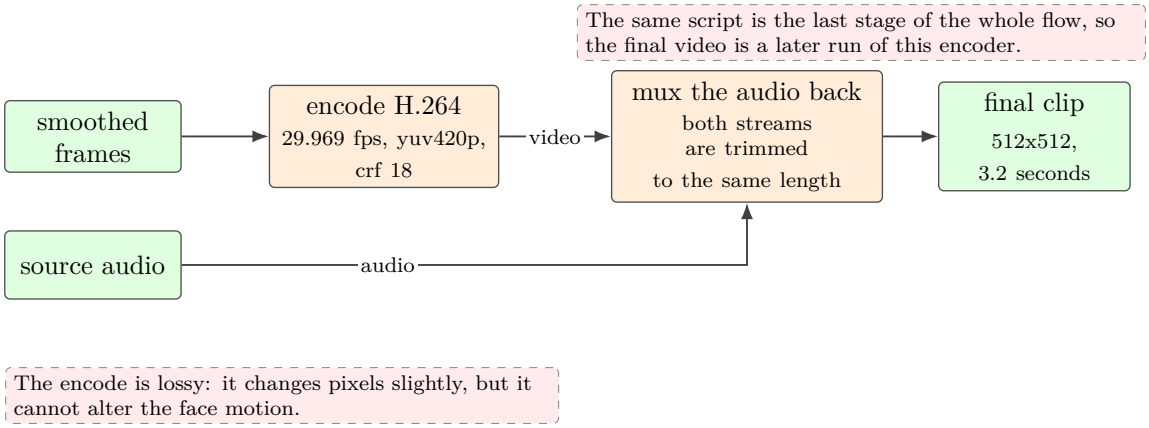


Figure 11: The reassembly stage. The smoothed frames are encoded as H.264 at the source rate and the original audio is muxed back; both streams are trimmed to the same length.

Stage	Face change (mean)	Face change (p95)	Consecutive identity	Identity to the portrait	Hair noise
Control (unswapped source footage)	6.6565	12.3488	0.9852	−0.0356	31.5999
Raw head swap	7.4510	17.3794	0.9228	0.2153	32.5703
Restoration	7.1602	16.2097	0.9273	0.2153	32.0721
Flow on the raw frames	6.6663	15.6581	0.9391	0.2148	32.0603
Restoration then flow, strength 0.3 (recommended)	6.6303	15.2802	0.9377	0.2152	31.7306
Deflicker (alternative)	5.6269	12.6650	0.9357	0.2106	31.8960

Table 6: All measured variants on the same 96 frames.



body input | raw swap | after restoration | after smoothing

Figure 12: The same frame 0175 at four points of the flow; the four images are all 512x512.

more change removed than the source holds, but motion, lighting, texture change and flicker all enter the number, so the drop alone does not prove that the smoother removed real motion. The control row is the unswapped source footage.

The identity-to-portrait value of about 0.215 sits above the control value of −0.0356. The unswapped footage scores near zero, and every swapped variant scores positive. The output is closer to the reference portrait, but the measure has no calibrated threshold, so this value alone does not prove identity transfer. Consecutive identity, about 0.938 on the recommended set, measures temporal similarity, and a wrong identity that stays consistent also scores high. The recommended set held up well against the control on the mean values. Figure 12 shows the effect of each stage on one frame.

	Autoencoder	FOMM	Rectified flow
What it does	One shared encoder with one decoder per person; a swap means the encoder of one person and the decoder of the other.	Animates a source photo with 10 keypoints, one Jacobian per keypoint, a motion field, and an occlusion map.	Head swap with a released rectified flow model and a LoRA adapter in four sampling steps, then restoration, smoothing and reassembly.
What it needs	238 prepared face images, 100 of Person A and 138 of Person B, at 128 × 128.	A source portrait of 512 × 512 and a driving clip of 256 × 256 with 320 frames at 29.97 FPS.	A 256 × 256 selfie portrait, 512 × 512 body frames, a fixed prompt of 409 characters, and one laptop with a 4 GB GPU and 8 GB of RAM.
What it produced	Both directions (A → B and B → A) after 50 and 150 epochs. The loss was low, with MSE 0.002935 and 0.001866 at epoch 150, but the output face did not take the other identity. The pretrained FaceFusion models looked better.	A 256 × 256 clip of 320 frames and 10.7 s in about 2.5 minutes on CPU. Head turns and blinks transferred smoothly, but braces disappeared in some frames.	A 96-frame 512 × 512 clip of 3.203 s with the original audio. The identity-to-portrait value was about 0.215 against −0.0356 for the control.
Main limitation	The reconstruction objective teaches the model to rebuild the input, not to transfer an identity, and the dataset was small.	Inpainting falls back on learned appearance priors, so rare personal details are lost and large head turns blur. No state passes from frame to frame.	Every frame is generated on its own, so the face boils; smoothing reduces the boil, but the worst frames still jump.

Table 7: The three methods: what each one does, its input, its result in this project, and its main limitation.

4.11 Problems and decisions

Attaching the adapter before the first forward pass cost 46 to 48 seconds per frame. Attaching it after one plain pass cost 16 seconds per frame. Peak VRAM was the same in both orders.

We standardised on the fast order. The internal cause was not established, but the effect reproduces exactly and explains an earlier timing discrepancy.

One fixed seed covers the whole clip, so the noise input is constant. The boil still happens, because the model is sensitive to small input changes. Only a pass across frames reduces it.

4.12 Limits

The results table shows a mean at the control level, but the tail stays high, so the picture still jumped on some frames. Each step works inside a frame or a short window, so no step removed that tail.

5 Comparison

The three methods attacked the same task from different sides. Table 7 lists what each one does, what it needs as input, what it produced in this project, and its main limitation.

The table shows three different trade-offs. The autoencoder needed our own training data and still missed the identity transfer, and our own model stayed behind the pretrained FaceFusion models. FOMM gave a fast per-frame method and moved head turns and blinks smoothly, but its inpainting lost rare details and it keeps no memory across frames. The rectified flow pipeline cost 16.3 seconds per frame in the generation stage, and the worst frames still jump,

yet it produced a full 512 × 512 clip with the original audio on the hardware we had. We would recommend that pipeline, with restoration followed by optical-flow smoothing at strength 0.3. Its identity-to-portrait value stayed above the control level, and the full clip ran on one laptop with a 4 GB GPU.

6 Ethics and Limitations

6.1 Ethics

Face reenactment carries a clear risk of misuse. The same pipeline that makes a light-hearted clip can put a real face on words the person never said, and a viewer cannot separate the two cases by eye. The risk grows as soon as the material leaves the team.

We answer that risk with provenance, with watermarks and with detection research. The technical measures limit what a stranger can do with a published file.

C2PA is a signed provenance and edit-history manifest. It records which tool made a file, what changed it and when, and a signature lets a reader check that record. It is not a stamp that exists only at capture time, because an editor can add a further manifest entry for each change, and the history then travels with the file. C2PA is also not a deepfake detector by itself. The manifest reports where a file came from and how it changed; it does not judge whether the content is real.

Watermarking is a separate measure. It hides a signal inside the pixels or the audio, and the signal travels with the content. A watermark and a C2PA manifest do different jobs, so a project can use both.

Research also reports detection cues. Eye blinking follows a rhythm that the early face generators did not reproduce (Li et al., 2018). Remote photoplethysmography reads a heartbeat signal from small colour changes in the skin (Qi et al., 2020). Frequency analysis finds the traces that neural upsampling leaves in the image spectrum (Frank et al., 2020). We treat these cues as research examples only. This project ran no detection experiment and trained no detector, so we report no measurement of our own here.

6.2 Limitations

Our results rest on artifacts that are not all in the repository. The autoencoder training code and the trained weights are absent, so that training cannot be repeated from this checkout. The Rectified Flow raw run records, the metric CSV and the 96-frame sets are absent as well, so the numbers in Section 4 cannot be re-derived here. The FOMM checkpoint is absent too, so the training history of those weights cannot be verified; the released configuration trains no adversarial loss, as Section 3 states.

Two explanations stay unverified. We ran no analysis of the dataset content, so the braces explanation in Section 3 stays a hypothesis and not a finding. The temporal metric thresholds are not calibrated, so a face change value has no absolute reading; only the control level gives it a meaning.

One helper limits what one number can show. The script that reports the mean absolute difference converts both images to 8-bit pixel values before the subtraction, an 8-bit round trip. The value therefore cannot show a difference finer than one 8-bit level.

These limits do not change any measured value. They set the size of each claim we make, and we report only what the artifacts support.

7 Conclusion

The three methods left us with three different results. FOMM moved head turns and blinks smoothly and produced its clip in about 2.5 minutes, but it lost rare details such as braces when

No.	Date	Mode	Attendees	Decisions
1	**3.10.2026**	**Online**	**Yusuf, Erdem, Mehmet**	**Yusuf, Erdem and Mehmet works on GAN, Autoencoder and Diffusion respectively.**
2	**10.10.2026**	**Online**	**Yusuf, Erdem, Mehmet and Eren**	**Discussion of findings and division of tasks for report and presentation creation.**

Table 8: The team meetings.

it inpainted the mouth. The rectified flow pipeline produced a 512 × 512 clip with sound on the 4 GB laptop we had, and its restoration and smoothing stages removed much of the boil. Our own autoencoder rebuilt faces well, but it never learned to move an identity from one person to the other.

The failures had reasons, or at least clear hypotheses. The autoencoder loss went down while the swap stayed wrong, because the training objective rewarded reconstruction and our dataset of 238 images was small; 100 more epochs did not change the result. The lost braces on FOMM have a hypothesis and no direct evidence: the training videos may show no braces, so the learned appearance priors cannot rebuild them. In the rectified flow clip every frame is generated on its own, so the face boils, and the smoothing only works inside short windows; the worst frames still jump. The newer adapter v1.1 was twelve times slower than v1, so the recorded clip kept v1.

What we learned reaches past the single results. A low loss does not mean a usable face swap, so the measure must match the goal. A model built for the task, even a released checkpoint, gave more than our own simple model. Per-frame generation always boils, and only a pass across frames reduces it. Finally, every measure here has limits: the face change value mixes motion, lighting, texture and flicker, and it has no calibrated threshold, so we read the numbers together with the frames, not instead of them.

8 Meetings

We couldn't arrange an in-person meeting so both meetings are held online as listed in Table 8.



Figure 13: The first team meeting.



Figure 14: The second team meeting.

The team divided the work by method, and each member wrote the part of the report that covers that method. **Erdem** built and trained the autoencoder, **Yusuf** ran the FOMM experiment, **Mehmet** ran the rectified flow pipeline, and **Eren** wrote the shared chapters and the slides.